Subject: OPTIMIZATION TECHNIQUES

Subject Code: 23AS2402e

UNIT III : Sequencing

Syllabus: Job sequencing, Johnson's algorithm for n jobs and two machines, n jobs and three machines, n jobs through m machines, two jobs through m machines.

I. Sequencing - Introduction

Definition:

Sequencing problem: n jobs have to be processed on m machines in a given technological order; we must find the order (sequence) of jobs that optimises a measure of performance - usually minimum ***total elapsed time***.

- Total elapsed time = time between the start of the first job on the first machine and the completion of the last job on the last machine.
- ***Idle time*** on a machine = time the machine is free between finishing one job and starting the next.
- Number of possible sequences for n jobs on m machines = $(n!)^m$. For 5 jobs on 2 machines: 120^2 - too many to enumerate.

Assumptions

- Each machine processes only one job at a time; each job on one machine at a time.
- Processing times are known and independent of the sequence.
- Once started a job is completed (no pre-emption); transfer time between machines is negligible.
- The order of machines for every job is the same (A then B then C ...) - for the n × m cases.

- Machines are ready and no breakdowns occur.

Notation

- t_{ij} = processing time of job i on machine j.
- Total elapsed time T = (finish time of last job on last machine).
- Idle time on last machine = T - Σ processing times on it (if the machine starts at 0).

> **Note:**
> Problem types in this unit: (a) n jobs, 2 machines; (b) n jobs, 3 machines; (c) n jobs, m machines; (d) 2 jobs, m machines.

2. Johnson's Algorithm - n Jobs, Two Machines

> **Definition:**
> Johnson's rule (S. M. Johnson, 1954) gives the optimal sequence minimising total elapsed time for n jobs through two machines M1 → M2 (every job first on M1, then on M2).

Algorithm

- Step 1: List the processing times of all jobs on M1 and M2.
- Step 2: Find the ***smallest*** time among all the times on both machines.
- Step 3: If it is on ***M1***, place that job at the ***first*** available position of the sequence; if it is on ***M2***, place it at the ***last*** available position.
- Step 4: Delete that job and repeat for the remaining jobs. Ties: break arbitrarily (ties between M1 and M2: put at M1 side, i.e. first).
- Step 5: Prepare the schedule, find total elapsed time and idle times.

> **Example:**
> Five jobs on two machines. Find the optimal sequence and elapsed time.

Job	1	2	3	4	5
M1	5	1	9	3	10
M2	2	6	7	8	4

- Smallest = 1 (Job 2 on M1) ⇒ first position: 2 _ _ _ _
- Next smallest = 2 (Job 1 on M2) ⇒ last position: 2 _ _ _ 1
- Next = 3 (Job 4 on M1) ⇒ second position: 2 4 _ _ 1
- Next = 4 (Job 5 on M2) ⇒ fourth position: 2 4 _ 5 1
- Remaining Job 3 fills the gap ⇒ **Optimal sequence: 2 - 4 - 3 - 5 - 1.**

Time table

Job	M1 in	M1 out	M2 in	M2 out
2	0	1	1	7
4	1	4	7	15
3	4	13	15	22
5	13	23	23	27
1	23	28	28	30

- Total elapsed time = **30** hours.
- Idle time M1 = 30 - 28 = **2**; Idle time M2 = 1 + 0 + 1 + 1 = **3** (check: 27 + 3 = 30 ✓).

> **Note:**
> A job on M2 starts at max(its finish on M1, finish of previous job on M2).

3. Another n Jobs, Two Machines Example

Example:

Six jobs; find the optimal sequence.

Job	A	B	C	D	E	F
M1	3	12	15	6	10	11
M2	8	10	10	6	12	1

- Smallest 1 (F on M2) ⇒ last: _ _ _ _ _ F
- Next 3 (A on M1) ⇒ first: A _ _ _ _ F
- Next 6 (D on M1 and D on M2 tie) ⇒ D on M1 first available: A D _ _ _ F
- Next 10 (B on M2, C on M2, E on M1): E on M1 ⇒ A D E _ _ F
- Remaining B (12, 10), C (15, 10): smallest 10 is on M2 for both: place B at last available (fifth), then C at fourth.
- **Sequence: A - D - E - C - B - F.**

Job	M1 in-out	M2 in-out
A	0 - 3	3 - 11
D	3 - 9	11 - 17
E	9 - 19	19 - 31
C	19 - 34	34 - 44
B	34 - 46	46 - 56
F	46 - 57	57 - 58

- Total elapsed time = **58**; idle M1 = 1; idle M2 = 3 + 2 + 3 + 2 + 1 = 11 (check: 47 + 11 = 58 ✓).

4. n Jobs and Three Machines

- Direct Johnson's rule works only for two machines. For three machines M1, M2, M3 we convert the problem into two fictitious machines G and H *if* one of

these conditions holds:

Formula:

Condition 1: $\min t_{i1} \geq \max t_{i2}$ OR Condition 2: $\min t_{i3} \geq \max t_{i2}$

- (the middle machine M2 is dominated by M1 or M3.)

Formula:

Fictitious machines: $G_i = t_{i1} + t_{i2}$, $H_i = t_{i2} + t_{i3}$

- Apply Johnson's rule to (G, H) to get the sequence; then compute the elapsed time using the ***original three machines***.
- If neither condition holds this method may not be optimal (use other methods).

Example:

Five jobs, three machines. Find the sequence and total elapsed time.

Job	1	2	3	4	5
M1	8	10	6	7	11
M2	5	6	2	3	4
M3	4	9	8	6	5

- $\min M1 = 6 \geq \max M2 = 6$ ✓ (Condition 1 satisfied).

Job	1	2	3	4	5
G = M1 + M2	13	16	8	10	15
H = M2 + M3	9	15	10	9	9

- Smallest = 8 (Job 3 on G) ⇒ first. Next 9 (H of jobs 1, 4, 5 – ties): place Job 1 last, Job 4 next from last, Job 5 next.
- Remaining Job 2 goes to position 2 ⇒ **Sequence: 3 – 2 – 5 – 4 – 1.**

Time table on the original machines

Job	M1 in-out	M2 in-out	M3 in-out
3	0 - 6	6 - 8	8 - 16
2	6 - 16	16 - 22	22 - 31
5	16 - 27	27 - 31	31 - 36
4	27 - 34	34 - 37	37 - 43
1	34 - 42	42 - 47	47 - 51

- **Total elapsed time = 51.** Idle: M1 = 51 - 42 = 9; M2 = 51 - 20 = 31; M3 = 51 - 32 = 19 (starting idle 8 included).

Note:

Lower bound check: Σ M1 + (M2 + M3 of last job) = 42 + 5 + 4 = 51 equals the answer, so the sequence is optimal.

5. n Jobs through m Machines

- Same idea as three machines: the order of machines is the same for all jobs, and the middle machines are "dominated".

Formula:

Conditions: $\min t_{i1} \geq \max t_{ij}$ (j = 2, ..., m-1) OR $\min t_{im} \geq \max t_{ij}$ (j = 2, ..., m-1)

- Convert to two fictitious machines:

Formula:

$G_i = t_{i1} + t_{i2} + \ldots + t_{i,m-1}$ (first m - 1 machines)
$H_i = t_{i2} + t_{i3} + \ldots + t_{im}$ (last m - 1 machines)

- Apply Johnson's rule on (G, H) to get the sequence; then compute the elapsed time on the original m machines.

Example:

Four jobs on four machines: A, B, C, D.

Job	1	2	3	4
A	13	8	9	10
B	5	3	4	6
C	4	5	3	2
D	12	9	10	11

- min A = 8, min D = 9; max of B = 6, max of C = 5. min A = 8 ≥ 6 ✓ (and 8 ≥ 5) - condition satisfied.

Job	1	2	3	4
G = A + B + C	22	16	16	18
H = B + C + D	21	17	17	19

- Smallest 16 (G of job 2 and job 3): put at first positions (job 2 first, job 3 second); next 17 (H of jobs 2, 3 - already placed); then Job 4 (18, 19): smallest is 18 on G ⇒ third; Job 1 (22, 21): last.
- **Sequence: 2 - 3 - 4 - 1.**

Job	A	B	C	D
2	0 - 8	8 - 11	11 - 16	16 - 25
3	8 - 17	17 - 21	21 - 24	25 - 35
4	17 - 27	27 - 33	33 - 35	35 - 46
1	27 - 40	40 - 45	45 - 49	49 - 61

- **Total elapsed time = 61.**

6. Two Jobs through m Machines (Graphical Method)

- Used for 2 jobs processed on m machines, each job with its own technological order (orders may differ).

Algorithm

- Step 1: Draw axes: horizontal axis = processing of Job 1 (machine times in order), vertical axis = processing of Job 2 (in its order).
- Step 2: For every machine draw the ***rectangle (block)*** formed by the intervals of Job 1 and Job 2 on that machine (both jobs cannot use it simultaneously).
- Step 3: Draw a path from the origin O (0, 0) to the end point (all of Job 1, all of Job 2) avoiding the interior of the blocks.
  - Move ***horizontally*** (only Job 1 works, Job 2 waits), ***vertically*** (only Job 2 works), or at ***45° diagonally*** (both work).
- Step 4: Choose the path with the ***maximum diagonal length*** (minimum horizontal and vertical movement).

Formula:

Total elapsed time = (total time of Job 1) + (sum of vertical segments) = (total time of Job 2) + (sum of horizontal segments)

Example:

Job 1: A (3), B (4), C (2). Job 2: B (2), A (5), C (3). Find minimum elapsed time.

Job 1 interval	A: 0 - 3	B: 3 - 7	C: 7 - 9
Job 2 interval	B: 0 - 2	A: 2 - 7	C: 7 - 10

- Blocks (x = Job 1, y = Job 2): A: (0 - 3) × (2 - 7); B: (3 - 7) × (0 - 2); C: (7 - 9) × (7 - 10).
- Best path: (0,0) → (2,2) diagonal → (3,2) horizontal 1 → (7,6) diagonal → (8,7) diagonal → (9,7) horizontal 1 → (9,10) vertical 3.

- Horizontal = 1 + 1 = 2; vertical = 3.
- Elapsed time = 9 + 3 = 12 = 10 + 2 ✓ ⇒ ***Minimum total elapsed time = 12 hours.***
- Interpretation: Job 2 waits 1 hour for machine A and 1 hour for machine C; Job 1 finishes at 9, Job 2 at 12.

> ***Note:***
> Blocks must never be crossed; diagonal moves along block edges are allowed.

7. Practice Problems with Answers

- 1) Two machines, 4 jobs: M1 = 4, 7, 3, 6; M2 = 5, 4, 8, 2. Find the sequence.
  - Ans: smallest 2 (job 4, M2) last; 3 (job 3, M1) first; 4 (job 1 on M1 and job 2 on M2): job 1 second, job 2 third ⇒ 3 – 1 – 2 – 4.
  - Times: job 3: M1 0–3, M2 3–11; job 1: M1 3–7, M2 11–16; job 2: M1 7–14, M2 16–20; job 4: M1 14–20, M2 20–22 ⇒ elapsed time 22.
- 2) For three machines with M1 = 3, 8, 7, M2 = 3, 4, 2, M3 = 5, 8, 6 – check the conditions. Ans: min M1 = 3 < max M2 = 4; min M3 = 5 ≥ 4 ✓ (condition 2) ⇒ G = 6, 12, 9; H = 8, 12, 8.
- 3) What is meant by idle time and total elapsed time?
- 4) State the assumptions of the sequencing problem.
- 5) How many sequences exist for 4 jobs on 3 machines? Ans: $(4!)^3$ = 13,824.
- 6) Why is Johnson's rule not directly applicable to 3 machines? Ans: M2 can become a bottleneck; conditions are needed to reduce to two machines.

> ***Note:***
> Always verify the elapsed time by a time table – do not trust only the sequence.

Quick Revision - Unit III

- Johnson: smallest time on M1 → first; on M2 → last; repeat.
- 3 machines: need min M1 ≥ max M2 or min M3 ≥ max M2; G = M1 + M2, H = M2 + M3.
- m machines: G = sum of first m - 1, H = sum of last m - 1.
- Idle time = elapsed time - total processing on that machine.
- Two jobs: graphical, elapsed time = total job time + vertical (or horizontal) movement.